

Felix Guzman

06/28/2026

Assignment: CS 470 Final Reflection

Presentation Video: https://www.youtube.com/watch?v=dNtf_Jtdlm0

Experiences and Strengths:

Before CS 470, my development experience regarding full stack web development mostly centered around DigitalOcean, MySQL/PHP, and some MEAN stack fundamentals. I was comfortable building and deploying full-stack applications, but I had limited exposure to Amazon Web Services and other cloud-native architecture options. This course, however, changed that significantly.

Specifically, this course directly contributes to advancing my professional goals by giving me real, hands-on experience with AWS, one of the most in-demand skill sets in cloud and full-stack development today. As someone around the tech-space, I consistently heard about AWS and other technologies I tackled in this class but never took the time to learn about them. In this class, I was able to do just that, as well as develop skills with containerization using Docker and Docker Compose, cloud storage with Amazon S3, serverless computing with AWS Lambda and API Gateway, NoSQL database management with Amazon DynamoDB, and cloud security with AWS Identity and Access Management. Technologies that I often heard about or saw in job postings but never experimented with myself. This is not just a personal anecdote, however, as according to our weekly resources, particularly in the Simplilearn video, we learn that these are the exact tools cloud employers look for. They highly value experience ranging from Docker and container knowledge for positions like Cloud Automation Engineers to AWS platform experience and cloud security for Cloud Architects.

As a developer, I have developed a problem-solving skillset that incorporates the typical methodical approach to problem-solving. Additionally, as a tech enthusiast, I am able to present my findings to different audiences with clear communication and speech that does not contain too much unnecessary jargon. Not to mention, I can be quite creative and elaborate on ideas and make them in a more personalized and efficient manner keeping the user in mind as a priority. Most importantly though, throughout my life, my strongest trait as well as weakness is my perfectionist nature, and relentless persistence to finish what I started. This is what makes it possible to get through difficult debugging sessions no matter how long they take. Interestingly enough, these qualities were made quite apparent in the final project, where getting Lambda, API Gateway, CORS, and DynamoDB to all work together required careful, step-by-step troubleshooting. The roles I am now prepared to pursue or learn more about would definitely include cloud developer, full-stack developer with AWS experience, and junior solutions

architect. These are all enabled as a result of having completed a full migration from a locally-hosted MEAN application to a serverless AWS workflow.

Planning for Growth:

Having migrated the QnA (theoretical startup) application into a fully serverless AWS architecture, I can speak to several approaches when planning for future growth.

Looking at what this course covered, serverless and microservices architecture offer three concrete efficiencies for managing and scaling an app like QnA going forward. Each Lambda function can be deployed or updated independently without touching the rest of the application, a failure in one function stays contained rather than bringing the whole system down, and each function scales on its own based on how much it is actually being used rather than scaling the entire application at once. As Cloud Bart from our weekly resources described, as developers we are responsible for "using the API Gateway or using AWS Lambda to glue things together" across services (Cloud Bart, 2020). This is something we did with QnA's six independent Lambda functions. Essentially, Lambda handles growing demand automatically without any changes on my end; for error handling, AWS CloudWatch tracks what each function is doing and any calls that do not go through are caught and retried rather than lost. All important for scale.

In regards to cost, it can be predicted in the serverless model by looking at how many times a function is called and how long it runs each time, since that is exactly what AWS charges for. This is not always the case in every cloud model. I first ran into this with a DigitalOcean project, where I paid for server capacity whether or not anyone was using the site. That idle cost is non-existent in AWS's serverless model. To summarize, serverless is the more cost-predictable option for a growing application because you only pay for what is actually used, whereas containers require paying for resources whether or not the traffic is justified. This all depends on your specific application, as for some models with consistently high and steady traffic, Amazon's container services can become more economical over time.

For expansion, the main advantages of serverless are no infrastructure to manage, automatic scaling, and costs that grow with the application rather than ahead of it; the main disadvantages are a brief delay the first time a function runs after sitting idle for a while, and the fact that Lambda functions do not hold onto information between calls, which requires some extra planning for anything that needs to remember state. That said, based on my own experience working across both models, serverless is the right fit for an app like QnA at this stage, with containers worth modifying only if demand becomes large and predictable enough to justify the added overhead.

Finally, we have elasticity and the pay-for-use model, which makes an excellent addition to serverless making it the strong industry standard it is today. Basically, instead of committing to a fixed hardware cost upfront and hoping traffic justifies it, the cloud adjusts to actual demand, meaning an app can scale from an early-stage project to a production application without over-investing ahead of growth. As Cloud Bart puts it, "the tools we use, the way we implement

them, and the way that we purchase and maintain them should drive business value before anything else" (Cloud Bart, 2020). This is how cloud models can improve businesses.

Overall, I've learned a lot during my time in CS-470 with the hands-on assignments and utilization of different technologies as I debugged, followed instructions, and got practical output, as well as with the quizzes and DBQs that helped me strengthen my practical knowledge. Interacting with my peers and receiving feedback was also essential. Even looking back on it all and writing this reflection has been helpful. My time in this class has most definitely been an investment and has become one more step as I move forward with my Computer Science and Software Development journey.

References:

Cloud Bart. (2020, February 26). AWS Architect, SysOps, or Developer - Which job is right for me? YouTube. <https://www.youtube.com/watch?v=45K3rPSSZ4>

Simplilearn. (2020, March 19). Top Job Roles In The Field Of Cloud Computing | Cloud Computing Job Opportunities | Simplilearn. YouTube. <https://www.youtube.com/watch?v=qGOKYcfeiYc>